

Enhancing Quantum Software Development Process with Experiment Tracking

Mahee Gamage
University of Jyväskylä
Jyväskylä, Finland
mahee.s.hewagamage@jyu.fi

Otso Kinanen
University of Jyväskylä
Jyväskylä, Finland
otso.j.r.kinanen@jyu.fi

Jake Muff
Quantum Algorithms and Software
VTT Technical Research Centre of Finland
Espoo, Finland
jake.muff@vtt.fi

Vlad Stirbu
University of Jyväskylä
Jyväskylä, Finland
vlad.a.stirbu@jyu.fi

Abstract—As quantum computing advances from theoretical promise to experimental reality, the need for rigorous experiment tracking becomes critical. Drawing inspiration from best practices in machine learning (ML) and artificial intelligence (AI), we argue that reproducibility, scalability, and collaboration in quantum research can benefit significantly from structured tracking workflows. This paper explores the application of MLflow in quantum research, illustrating how it enables better development practices, experiment reproducibility, decision making, and cross-domain integration in an increasingly hybrid classical-quantum landscape.

I. INTRODUCTION

Quantum software engineering is an emerging discipline fueled by recent advances in quantum hardware. Despite being part of the broader field of software engineering, quantum software engineering has to take into account the current and near-term technical limitations of the Noisy intermediate-scale quantum (NISQ) [3] era hardware. The limited number of qubits available in quantum processing units (QPUs) and their quality have a direct impact on the complexity of the programs that can be executed, highlighting important challenges [2], which have to be addressed to effectively leverage the promises of quantum computing.

A quantum software or algorithm developer must navigate this complicated environment in a structured way following a well-defined software development lifecycle [5], supported by specialized tools [1]. Given the limited availability of quantum hardware, the development process begins on quantum simulators. Subsequently, as the program or algorithm matures, its execution is performed on the actual hardware. Due to the unique nature of quantum environments, developers must collect relevant data to guide each development stage. Early on, the focus is on the program or algorithm itself, but once executed on a QPU, data collection expands to include hardware-specific information such as calibration and qubit quality.

In this paper, we propose a quantum experiment tracking system built on MLflow, an open-source platform widely

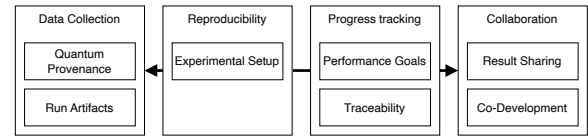


Fig. 1. Quantum software development activities supported by experiment tracking

adopted in ML/AI development. By leveraging its existing capabilities, we focus on developing quantum-specific extensions while benefiting from its mature ecosystem and the operational expertise of an already skilled workforce.

II. BACKGROUND AND MOTIVATION

Experiment tracking [7] was introduced as a tool that addresses the challenges faced by machine learning (ML) developers in the following four areas: multitude of tools, experiment tracking, reproducibility, and production deployment. We can observe that quantum software development has some similarities with ML/AI development. For example, developers have the option to use several quantum software development toolkits (e.g. Qiskit¹, PennyLane², Qrisp³, etc.) that enables them to interact with a multitude of hardware vendors in order to execute their quantum routines. During this process, developers have to collect a plethora of software and hardware related data to steer the software development activities in the right direction.

Experiment tracking fundamentally involves collecting and logging data about the software and hardware used. QProv [6] introduces a quantum provenance model with a data schema that captures four key categories: quantum circuit, quantum computer, compilation, and execution. These form the core of experiment tracking and can be extended with application-specific artifacts. Given the experimental nature of current quantum hardware, run data is essential not only for evaluating performance but also for ensuring reproducibility,

This work has been supported by the Academy of Finland (project DE-QSE 349945), Business Finland (EM4QS 155/31/2024), Finnish Ministry of Education and Culture through the Quantum Doctoral Education Pilot Program (QDOC VN /3137/2024-OKM-4) and the Research Council of Finland through Finnish Quantum Flagship project (359240, JYU).

¹<https://www.ibm.com/quantum/qiskit>

²<https://pennylane.ai>

³<https://www.qrisp.eu>

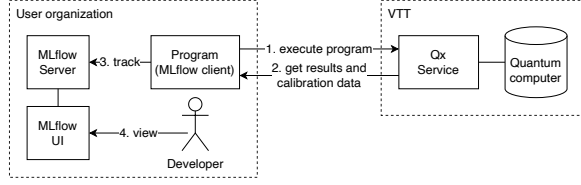


Fig. 2. Experiment setup: the user executes a program on the on a quantum computer exposed via the VTT QX service and tracks the experiment results to the MLflow tracking server operated by its own organization

```

1 import mlflow
2 mlflow.set_experiment("Qx VTT Demo for QCE")
3 with mlflow.start_run():
4     mlflow.set_tag("Training info", "Qiskit on Qx")
5     provider = IQMProvider("https://qx.vtt.fi/api/devices/q50")
6     backend = provider.get_backend()
7     shots = 500
8     result = demo_function(backend, shots)
9     mlflow.log_param("shots", shots)
10    mlflow.log_figure(plot_histogram(result.get_counts()),
11                      "results.png")
12    calibration_set_id = str(result.results[0].calibration_set_id)
13    mlflow.log_text(calibration_set_id, "calibration_set_id.txt")
14    calibration_data = get_calibration_data(backend.client,
15                                          calibration_set_id)
16    mlflow.log_dict(calibration_data, "calibration_data.json")
  
```

Listing 1. Tracking experiment data with MLflow. The implementation of `demo_function`, `plot_histogram` and `get_calibration_data` functions has been omitted for brevity.

since hardware reliability can vary between runs, and simulators cannot fully replace real hardware experimentation [4]. Additionally, the collected data supports progress tracking by providing a structured view of how performance goals are met, helping guide development to subsequent stages [1]. These artifacts also offer traceability and support decision-making. Finally, since quantum software development is inherently collaborative, experiment tracking enhances result sharing and supports joint development between hardware and software teams, as depicted in Fig. 1.

III. QUANTUM EXPERIMENT TRACKING

Since its introduction, MLflow⁴ has become a standard tool for managing the development lifecycle of ML projects, including more complex domains like Generative AI and Large Language Models. In our study, we used MLflow to track quantum experiment data from programs executed on an IQM 50-qubit quantum computer, operated by VTT via the QX service. The user's organization hosts the MLflow tracking server and UI, while data is logged using the MLflow client integrated into the quantum program. The experiment setup is illustrated in Fig. 2.

Experiment tracking data collection is performed using the MLflow client library that is included in the quantum program. MLflow client can name experiments, set tags on individual runs, and log parameters, metrics, or free-form artifacts (e.g.

```

1 import mlflow
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 experiment_name = "Qx VTT Demo for QCE"
5 exp = mlflow.get_experiment_by_name(experiment_name)
6 runs_df = mlflow.search_runs(experiment_ids=[exp.experiment_id])
7 ...
8 # Data-frame can be further processed with pandas and matplotlib
  
```

Listing 2. Processing experiments data with MLflow

figures, datasets, etc), capabilities that can capture the QProv attributes. A simplified⁵ program is depicted in the Listing. 1.

MLflow client library can be used to process **data from multiple experiments** using the search functionality. The result of a query can be used with other popular libraries to create new artifacts that support decision making activities. A simplified program is depicted in the Listing. 2.

IV. DISCUSSION AND FUTURE WORK

In this paper, we demonstrated how MLflow can serve as a foundation for quantum experiment tracking. We showed that MLflow is well-suited to quantum research, supporting improved development practices, reproducibility, informed decision-making, and collaboration. Instead of building standalone solutions like the QProv provenance system [6] or Aqueduct⁶, we advocate for leveraging mature and widely adopted tools with strong community and ecosystem support. As next steps, we plan to investigate the collection of data from selected QPU providers according to the QProv attribute schema. By creating a library that wraps the experiment tracking functionality, we facilitate the integration of this practice into quantum software development workflows.

REFERENCES

- [1] O. Kinanen, A. D. Muñoz-Moller, V. Stirbu, J. M. Murillo, and T. Mikkonen. Toolchain for faster iterations in quantum software development. *Computing*, 107(4):99, Mar 2025.
- [2] J. M. Murillo, J. Garcia-Alonso, E. Moguel, J. Barzen, F. Leymann, S. Ali, T. Yue, P. Arcaini, R. Pérez-Castillo, I. García Rodríguez de Guzmán, M. Piattini, A. Ruiz-Cortés, A. Brogi, J. Zhao, A. Miranskyy, and M. Wimmer. Quantum software engineering: Roadmap and challenges ahead. *ACM Trans. Softw. Eng. Methodol.*, Jan. 2025.
- [3] J. Preskill. Quantum Computing in the NISQ era and beyond. *Quantum*, 2:79, Aug. 2018.
- [4] P. Senapati, Z. Wang, W. Jiang, T. S. Humble, B. Fang, S. Xu, and Q. Guan. Towards redefining the reproducibility in quantum computing: A data analysis approach on nisq devices. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 01, pages 468–474, Sep. 2023.
- [5] B. Weder, J. Barzen, F. Leymann, M. Salm, and D. Vietz. The quantum software lifecycle. In *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software, APEQS 2020*, page 2–9, New York, NY, USA, 2020. Association for Computing Machinery.
- [6] B. Weder, J. Barzen, F. Leymann, M. Salm, and K. Wild. Qprov: A provenance system for quantum computing. *IET Quantum Communication*, 2(4):171–181, 2021.
- [7] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, et al. Accelerating the machine learning lifecycle with mlflow. *IEEE Data Eng. Bull.*, 41(4):39–45, 2018.

⁴<https://mlflow.org>

⁵Full program available at <https://github.com/qubernetes-dev/q8s-examples>

⁶<https://github.com/AqueductHub>